

初心者向けツアーでは、コード内で `null` 値を扱う方法を学びました。この章では、ヌルセーフ機能の一般的な使用例と、それらを最大限に活用する方法について解説します。

Smart casts and safe casts

スマートキャストとセーフキャスト

Kotlinでは、明示的な宣言がなくても型を推論できる場合があります。Kotlinに対して、変数やオブジェクトを特定の型に属するものとして扱うよう指示することを、**型変換**と呼びます。型が自動的にキャストされる場合（型推論が行われる場合など）、それは**スマートキャスト**と呼ばれます。

is and !is operators

is および !is 演算子

型変換の仕組みについて詳しく見る前に、オブジェクトが特定の型であるかどうかを確認する方法を見てみましょう。For this, you can use the `when` または `if` の条件式と組み合わせて使用する `is` および `!is` 演算子：

- `is` は、オブジェクトが指定された型であるかどうかを調べ、ブール値を返します。
- `!is` は、オブジェクトがその型を**持っていない**かどうかを調べ、ブール値を返します。

For example:

```
fun printObjectType(obj: Any) {
    when (obj) {
        is Int -> println("It's an Integer with value $obj")
        !is Double -> println("It's NOT a Double")
        else -> println("Unknown type")
    }
}

fun main() {
    val myInt = 42
    val myDouble = 3.14
    val myList = listOf(1, 2, 3)

    // The type is Int
    printObjectType(myInt)
    // It's an Integer with value 42

    // The type is List, so it's NOT a Double.
    printObjectType(myList)
    // It's NOT a Double

    // The type is Double, so the else branch is triggered.
    printObjectType(myDouble)
    // Unknown type
}
```

```
{kotlin-runnable="true" id="kotlin-tour-null-safety-casts"}
```

`is` および `!is` 演算子と組み合わせて `when` 条件式を使用する方法の例については、[Open およびその他の特殊クラス](#) の章ですでに紹介されています。

```
{style="tip"}
```

as and as? operators

as および as 演算子

オブジェクトを他の型に明示的に_キャスト_するには、`as` 演算子を使用します。これには、Null許容型からNull非許容型への型変換も含まれます。キャストができない場合、プログラムは**実行時にクラッシュ**します。だからこそ、これは****安全でない**型変換演算子**と呼ばれるのです。**

```
fun main() {
    //sampleStart
    val a: String? = null
    val b = a as String

    // Triggers an error at runtime
    print(b)
    //sampleEnd
}
```

```
{kotlin-runnable="true" validate="false" id="kotlin-tour-null-safety-as-operator"}
```

オブジェクトを明示的にヌル不可型にキャストし、失敗時にエラーをスローするのではなく `null` を返すには、`as?` 演算子を使用します。`as?` 演算子は、失敗してもエラーを発生させないため、**安全な演算子**と呼ばれています。

```
fun main() {
    //sampleStart
    val a: String? = null
    val b = a as? String

    // Returns null value
    print(b)
    // null
    //sampleEnd
}
```

```
{kotlin-runnable="true" id="kotlin-tour-null-safety-safe-operator"}
```

`as?` 演算子をエルビス演算子 `?:` と組み合わせることで、数行のコードを1行にまとめることができます。たとえば、次の `calculateTotalStringLength()` 関数は、さまざまな種類の文字列が混在するリストに含まれるすべての文字列の合計長さを計算します：

```
fun calculateTotalStringLength(items: List<Any>): Int {
    var totalLength = 0

    for (item in items) {
        totalLength += if (item is String) {
            item.length
        } else {
            0 // Add 0 for non-String items
        }
    }

    return totalLength
}
```

例：

- `totalLength` 変数をカウンターとして使用します。
- `for` ループを使用して、リスト内のすべての項目を順に処理します。
- `if` ステートメントと `is` 演算子を使用して、現在の項目が文字列であるかどうかを確認します：
 - そうである場合、その文字列の長さがカウンタに加算されます。
 - そうでない場合、カウンタはインクリメントされません。
- `totalLength` 変数の最終的な値を返します。

このコードは次のように簡略化できます：

```
fun calculateTotalStringLength(items: List<Any>): Int {
    return items.sumOf { (it as? String)?.length ?: 0 }
}
```

この例では、`.sumOf()` 拡張関数を使用しており、次のようなラムダ式を指定しています：

- リスト内の各項目について、`as?` を使用して `String` への安全なキャストを実行します。
- 呼び出しが `null` 値を返さない場合に、安全な呼び出し `?.` を使用して `length` プロパティにアクセスします。
- エルビス演算子 `?:` を使用し、セーフコールが `null` 値を返した場合に `0` を返します。

ヌル値とコレクション

Kotlinでは、コレクションを扱う際、`null`値の処理や不要な要素の除外が必要になることがよくあります。Kotlinには、リスト、セット、マップ、その他のコレクションを扱う際に、クリーンで効率的かつnullセーフなコードを書くために利用できる便利な関数が用意されています。

リストから `null` 値をフィルタリングするには、`filterNotNull()` 関数を使用します：

```
fun main() {
    //sampleStart
    val emails: List<String?> = listOf("alice@example.com", null,
```

```
"bob@example.com", null, "carol@example.com")

    val validEmails = emails.filterNotNull()

    println(validEmails)
    // [alice@example.com, bob@example.com, carol@example.com]
//sampleEnd
}
```

{kotlin-runnable="true" id="kotlin-tour-null-safety-filterNotNull"}

リストを作成する際に、`null` 値のフィルタリングを直接行いたい場合は、`listOfNotNull()` 関数を使用してください：

```
fun main() {
//sampleStart
    val serverConfig = mapOf(
        "appConfig.json" to "App Configuration",
        "dbConfig.json" to "Database Configuration"
    )

    val requestedFile = "appConfig.json"
    val configFiles = listOfNotNull(serverConfig[requestedFile])

    println(configFiles)
    // [App Configuration]
//sampleEnd
}
```

{kotlin-runnable="true" id="kotlin-tour-null-safety-listOfNotNull"}

これらの例の両方において、すべての項目が `null` 値の場合、空のリストが返されます。

Kotlin には、コレクション内の値を検索するために使用できる関数も用意されています。値が見つからない場合、エラーを発生させるのではなく、`null` 値を返します：

- `maxOrNull()` は、最大値を求めます。最大値が存在しない場合は、`null` を返します。
- `minOrNull()` は、最小値を検索します。最小値が存在しない場合は、`null` を返します。

例えば：

```
fun main() {
//sampleStart
    // 1週間にわたって記録された気温
    val temperatures = listOf(15, 18, 21, 21, 19, 17, 16)

    // その週の最高気温を算出する
    val maxTemperature = temperatures.maxOrNull()
    println("Highest temperature recorded: ${maxTemperature ?: "No data"}")
}
```

```
// Highest temperature recorded: 21

// その週の最低気温を算出する
val minTemperature = temperatures.minOrNull()
println("Lowest temperature recorded: ${minTemperature ?: "No data"}")
// Lowest temperature recorded: 15
//sampleEnd
}
```

```
{kotlin-runnable="true" id="kotlin-tour-null-safety-collections"}
```

この例では、Elvis演算子 `?:` を使用して、関数が `null` 値を返した場合に、出力文を返すようにしています。

`maxOrNull()` および `minOrNull()` 関数は、`null` 値を含んでいないコレクションで使用するよう設計されています。そうしないと、関数が目的の値を見つけられなかったのか、それとも `null` 値を見つけたのかが判別できません。

```
{style="note"}
```

条件に一致する単一の項目を見つけるには、ラムダ式とともに `singleOrNull()` 関数を使用できます。該当するものが存在しない場合、または一致する項目が複数ある場合、この関数は `null` 値を返します：

```
fun main() {
//sampleStart
    // 1週間にわたって記録された気温
    val temperatures = listOf(15, 18, 21, 21, 19, 17, 16)

    // 30度の日がちょうど1日あったかどうかを確認する
    val singleHotDay = temperatures.singleOrNull{ it == 30 }
    println("Single hot day with 30 degrees: ${singleHotDay ?: "None"}")
    // Single hot day with 30 degrees: None
//sampleEnd
}
```

```
{kotlin-runnable="true" id="kotlin-tour-null-safety-singleornull"}
```

`singleOrNull()` 関数は、`null` 値を含んでいないコレクションで使用するよう設計されています。

```
{style="note"}
```

一部の関数は、ラムダ式を使用してコレクションを変換し、その目的を果たせない場合は `null` を返します。

ラムダ式を使用してコレクションを変換し、`null` ではない最初の値を返すには、`firstNotNullOfOrNull()` 関数を使用します。そのような値が存在しない場合、この関数は `null` 値を返します：

```
fun main() {
//sampleStart
    data class User(val name: String?, val age: Int?)

    val users = listOf(
```

```

        User(null, 25),
        User("Alice", null),
        User("Bob", 30)
    )

    val firstNonNullName = users.firstNotNullOfOrNull { it.name }
    println(firstNonNullName)
    // Alice
//sampleEnd
}

```

```
{kotlin-runnable="true" id="kotlin-tour-null-safety-firstnotnullofornull"}
```

ラムダ式を使用して、コレクションの各要素を順次処理し、累積値を生成する（または、コレクションが空の場合は `null` 値を返す）には、`reduceOrNull()` 関数を使用します：

```

fun main() {
//sampleStart
    // ショッピングカート内の商品の価格
    val itemPrices = listOf(20, 35, 15, 40, 10)

    // reduceOrNull() 関数を使用して合計金額を計算する
    val totalPrice = itemPrices.reduceOrNull { runningTotal, price -> runningTotal
+ price }
    println("Total price of items in the cart: ${totalPrice ?: "No items"}")
    // カート内の商品の合計金額：120

    val emptyCart = listOf<Int>()
    println(emptyCart) // emptyCartはnullの配列 [] ：空のカート （訳者加筆）
    val emptyTotalPrice = emptyCart.reduceOrNull { runningTotal, price ->
runningTotal + price }
    println("Total price of items in the empty cart: ${emptyTotalPrice ?: "No
items"}")
    // 空のカート内の商品の合計金額：商品はありません。
//sampleEnd
}

```

```
{kotlin-runnable="true" id="kotlin-tour-null-safety-reduceornull"}
```

この例では、関数が `null` 値を返した場合に、出力文を返すために Elvis 演算子 `?:` も使用しています。

`reduceOrNull()` 関数は、`null` 値を含んでいないコレクションで使用するよう設計されています。

```
{style="note"}
```

Kotlinの[標準ライブラリ](#)を調べて、コードをより安全にするために利用できる関数をさらに見つけてみましょう。

Early returns and the Elvis operator

初期の集計結果とエルヴィス・オペレーター

初心者向けツアーでは、[early returns\(関数内での処理の切上げ\)](#) を使って、関数の処理を特定のポイント以降に進まないようにする方法について学びました。関数内の前提条件を確認するには、エルビス演算子 `?:` と早期リターンを組み合わせて使用できます。この方法なら、ネストしたチェックを行う必要がないため、コードを簡潔に保つのに最適です。コードの複雑さが軽減されることで、メンテナンスも容易になります。例えば：

```
data class User(  
    val id: Int,  
    val name: String,  
    // この人がもつ友人のユーザーIDを記した一覧  
    val friends: List<Int>  
)  
  
// ユーザーの友達の数を取得する関数  
fun getNumberOfFriends(users: Map<Int, User>, userId: Int): Int {  
    // ユーザーを取得します。見つからない場合は -1 を返します。  
    val user = users[userId] ?: return -1  
    // 友達の数を出す  
    return user.friends.size  
}  
  
fun main() {  
    // Creates some sample users  
    val user1 = User(1, "Alice", listOf(2, 3))  
    val user2 = User(2, "Bob", listOf(1))  
    val user3 = User(3, "Charlie", listOf(1))  
  
    // Creates a map of users  
    val users = mapOf(1 to user1, 2 to user2, 3 to user3)  
  
    // usersの中から指定したIDの人に何人の友達がいるかを表示。指定したIDなusersの中に居  
    // ない場合は-1を表示  
    println(getNumberOfFriends(users, 1))  
    // 2  
    println(getNumberOfFriends(users, 2))  
    // 1  
    println(getNumberOfFriends(users, 4))  
    // -1  
}
```

{kotlin-runnable="true" id="kotlin-tour-null-safety-early-return"}

この例では：

- `User` というデータクラスがあり、ユーザーの `id`、`name`、および友達のリストを表すプロパティを持っています。
- `getNumberOfFriends()` 関数について：
 - `User` インスタンスのマップと、整数型のユーザー ID を受け取ります。

- 指定されたユーザー ID を持つ `User` インスタンスのマップの値を取得します。
- Elvis演算子を使用し、マップの値が `null` の場合、関数を早期に終了して値 `-1` を返します。
- マップから取得した値を `user` 変数に代入します。
- `size` プロパティを使用して、ユーザーのフレンドリストにあるフレンドの数を返します。
- `main()` 関数について:
 - `User` のインスタンスを3つ作成します。
 - これらの `User` インスタンスのマップを作成し、それを `users` 変数に代入します。
 - `users` 変数に対して、値 `1` と `2` を引数として `getNumberOfFriends()` 関数を呼び出し、「Alice」には2人の友達、「Bob」には1人の友達が返されます。
 - `users` 変数に対して `getNumberOfFriends()` 関数を呼び出し、値として `4` を指定すると、値 `-1` を返して早期に処理が終了します。

早期リターンを省けば、コードがもっと簡潔になることに気づくかもしれません。ただし、`users[userId]` が `null` を返す可能性があるため、このアプローチでは複数の `safe` 呼び出しが必要となり、コードの可読性が若干低下してしまいます：

```
fun getNumberOfFriends(users: Map<Int, User>, userId: Int): Int {  
    // ユーザーを取得する。見つからない場合は-1を返す  
    return users[userId]?.friends?.size ?: -1  
}
```

{validate="false"}

この例では、Elvis演算子を使って1つの条件のみをチェックしていますが、重大なエラー経路をすべて網羅するために、複数のチェックを追加することも可能です。Elvis演算子による早期評価により、`null` 値や無効なケースが検出されるとすぐに処理を停止するため、プログラムが不要な処理を行うのを防ぎ、コードの安全性を高めることができます。

コード内で `return` をどのように使用できるかについての詳細は、[戻り値とジャンプ](#) を参照してください。

Practice {completion-point="true"}

演習 {completion-point=「true」}

課題 1 {initial-collapse-state="collapsed" collapsible="true" id="null-safety-exercise-1"}

あなたは、ユーザーがさまざまな種類の通知を有効または無効にできるアプリ向けの通知システムを開発しています。 `getNotificationPreferences()` 関数を、以下の条件を満たすように完成させてください：

1. `validUser` 変数は、`as?` 演算子を使用して、`user` が `User` クラスのインスタンスであるかどうかを確認します。そうでない場合は、空のリストを返します。
2. `userName` 変数では、Elvis の `?:` 演算子を使用しており、ユーザー名が `null` の場合、デフォルトで「Guest」となるようにしています。
3. 最後のreturn文では、`.takeIf()` 関数を使用して、メールおよびSMS通知の設定が有効になっている場合にのみ、それらを含めるようにしています。
4. `main()` 関数は正常に実行され、期待通りの出力が表示されます。

`takeIf()` 関数は、指定された条件が真の場合、元の値を返し、そうでない場合は `null` を返します。例えば：

```
fun main() {
```

c > // ユーザーがログイン済みです

```
val userIsLoggedIn = true
// ユーザーのセッションが有効です
val hasSession = true

// ユーザーがログインしており、セッションが有効な場合にダッシュボードへのアクセスを許可する
val canAccessDashboard = userIsLoggedIn.takeIf { hasSession }

println(canAccessDashboard ?: "Access denied")
// true
```

```
}
```

{style = "tip"}

|--|--|

```
data class User(val name: String?)

fun getNotificationPreferences(user: Any, emailEnabled: Boolean, smsEnabled: Boolean): List<String> {
    val validUser = // Write your code here
    val userName = // Write your code here

    return listOfNotNull( /* Write your code here */ )
}

fun main() {
    val user1 = User("Alice")
    val user2 = User(null)
    val invalidUser = "NotAUser"

    println(getNotificationPreferences(user1, emailEnabled = true, smsEnabled = false))
    // [Email Notifications enabled for Alice]
    println(getNotificationPreferences(user2, emailEnabled = false, smsEnabled = true))
}
```

```
    // [SMS Notifications enabled for Guest]
    println(getNotificationPreferences(invalidUser, emailEnabled = true,
    smsEnabled = true))
    // []
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-null-safety-exercise-1"}

|--|--|

```
data class User(val name: String?)

fun getNotificationPreferences(user: Any, emailEnabled: Boolean, smsEnabled:
Boolean): List<String> {
    val validUser = user as? User ?: return emptyList()
    val userName = validUser.name ?: "Guest"

    return listOfNotNull(
        "Email Notifications enabled for $userName".takeIf { emailEnabled },
        "SMS Notifications enabled for $userName".takeIf { smsEnabled }
    )
    // 訳者注: listOfNotNullは、takeIfでその後の{}ブロック内の条件を評価し、trueな
    // らtakeIfの前の文字列をリストに加える。
}

fun main() {
    val user1 = User("Alice")
    val user2 = User(null)
    val invalidUser = "NotAUser"

    println(getNotificationPreferences(user1, emailEnabled = true, smsEnabled =
false))
    // [Email Notifications enabled for Alice]
    println(getNotificationPreferences(user2, emailEnabled = false, smsEnabled =
true))
    // [SMS Notifications enabled for Guest]
    println(getNotificationPreferences(invalidUser, emailEnabled = true,
smsEnabled = true))
    // []

    // 理解を深めるため訳者追記 (Bobがemailとsmsを持っている場合)
    val user3 = User("Bob")
    println(getNotificationPreferences(user3, emailEnabled = true, smsEnabled =
true))
    // [Email Notifications enabled for Bob, SMS Notifications enabled for Bob]
    // 追記終わり
}
```

```
{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-null-safety-solution-1"}
```

課題 2 {initial-collapse-state="collapsed" collapsible="true" id="null-safety-exercise-2"}

現在、ユーザーが複数のサブスクリプションを契約できるサブスクリプション型のストリーミングサービスの開発に取り組んでいますが、**一度に有効にできるのは1つだけです**。 `getActiveSubscription()` 関数を完成させ、アクティブなサブスクリプションが2つ以上ある場合に `null` 値を返すよう、述語を指定して `singleOrNull()` 関数を使用するようにしてください：

|--|--|

```
data class Subscription(val name: String, val isActive: Boolean)

fun getActiveSubscription(subscriptions: List<Subscription>): Subscription? //
Write your code here

fun main() {
    val userWithPremiumPlan = listOf(
        Subscription("Basic Plan", false),
        Subscription("Premium Plan", true)
    )

    val userWithConflictingPlans = listOf(
        Subscription("Basic Plan", true),
        Subscription("Premium Plan", true)
    )

    println(getActiveSubscription(userWithPremiumPlan))
    // Subscription(name=Premium Plan, isActive=true)

    println(getActiveSubscription(userWithConflictingPlans))
    // null
}
```

```
{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-null-safety-exercise-2"}
```

|--|--|

```
data class Subscription(val name: String, val isActive: Boolean)

fun getActiveSubscription(subscriptions: List<Subscription>): Subscription? {
    return subscriptions.singleOrNull { subscription -> subscription.isActive }
    // singleOrNullは、subscriptionで構成されたsubscriptionsのリストの中から
    // ひとつひとつのsubscriptionを抜き出し、isActiveが真であるものを拾い出し数え、
    // 真の数がひとつの時、そのsubscriptionを返す。真の数がひとつ以外ならnullを返す。
}
```

```

fun main() {
    // premium Planのみtrueの時
    val userWithPremiumPlan = listOf(
        Subscription("Basic Plan", false),
        Subscription("Premium Plan", true)
    )

    // basicPlanとpremiumPlanの両方がtrueの時
    val userWithConflictingPlans = listOf(
        Subscription("Basic Plan", true),
        Subscription("Premium Plan", true)
    )

    println(getActiveSubscription(userWithPremiumPlan))
    // Subscription(name=Premium Plan, isActive=true)

    println(getActiveSubscription(userWithConflictingPlans))
    // null
}

```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution 1" id="kotlin-tour-null-safety-solution-2-1"}

|--|--|

```

data class Subscription(val name: String, val isActive: Boolean)

fun getActiveSubscription(subscriptions: List<Subscription>): Subscription? =
    subscriptions.singleOrNull { it.isActive }

fun main() {
    val userWithPremiumPlan = listOf(
        Subscription("Basic Plan", false),
        Subscription("Premium Plan", true)
    )

    val userWithConflictingPlans = listOf(
        Subscription("Basic Plan", true),
        Subscription("Premium Plan", true)
    )

    println(getActiveSubscription(userWithPremiumPlan))
    // Subscription(name=Premium Plan, isActive=true)

    println(getActiveSubscription(userWithConflictingPlans))
    // null
}

```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution 2" id="kotlin-tour-null-safety-solution-2-2"}

課題 3 {initial-collapse-state="collapsed" collapsible="true" id="null-safety-exercise-3"}

あなたは、ユーザーがユーザー名とアカウントの状態を持つソーシャルメディアプラットフォームの開発に取り組んでいます。現在アクティブなユーザー名のリストを確認したい。 `getActiveUsernames()` 関数を完成させ、`mapNotNull()` 関数の述語が、ユーザー名がアクティブな場合はそのユーザー名を、そうでない場合は `null` 値を返すようにしてください：

|--|--|

```
data class User(val username: String, val isActive: Boolean)

fun getActiveUsernames(users: List<User>): List<String> {
    return users.mapNotNull { /* Write your code here */ }
}

fun main() {
    val allUsers = listOf(
        User("alice123", true),
        User("bob_the_builder", false),
        User("charlie99", true)
    )

    println(getActiveUsernames(allUsers))
    // [alice123, charlie99]
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-null-safety-exercise-3"}

|--|--|

Just like in Exercise 1, you can use the `takeIf()` function when you check if the user is active.

{ style = "tip" }

|--|--|

```
data class User(val username: String, val isActive: Boolean)

fun getActiveUsernames(users: List<User>): List<String> {
    return users.mapNotNull { user ->
        if (user.isActive) user.username else null
    }
}

fun main() {
    val allUsers = listOf(
        User("alice123", true),
        User("bob_the_builder", false),
        User("charlie99", true)
    )
}
```

```

    )

    println(getActiveUsernames(allUsers))
    // [alice123, charlie99]
}

```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution 1" id="kotlin-tour-null-safety-solution-3-1"}

|--|--|

```

data class User(val username: String, val isActive: Boolean)

fun getActiveUsernames(users: List<User>): List<String> =
    users.mapNotNull { user -> user.username.takeIf { user.isActive } }

fun main() {
    val allUsers = listOf(
        User("alice123", true),
        User("bob_the_builder", false),
        User("charlie99", true)
    )

    println(getActiveUsernames(allUsers))
    // [alice123, charlie99]
}

```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution 2" id="kotlin-tour-null-safety-solution-3-2"}

課題 4 {initial-collapse-state="collapsed" collapsible="true" id="null-safety-exercise-4"}

あなたは、あるEコマースプラットフォーム向けの在庫管理システムの開発に取り組んでいます。販売処理を行う前に、商品の注文数量が在庫状況に基づいて妥当であるかを確認する必要があります。

`validateStock()` 関数を完成させ、早期リターンと（適切な場合は）エルヴィス演算子を使用して、以下の条件を満たしているかどうかを確認するようにしてください：

- `requested` 変数が `null`
- `available` 変数は `null`
- `requested` 変数の値が負の値
- `requested` 変数の値が `available` 変数の値よりも大きい

上記のいずれの場合においても、関数は `-1` を返して早期に終了しなければなりません。

|--|--|

```

fun validateStock(requested: Int?, available: Int?): Int {
    // Write your code here
}

```

```
}

fun main() {
    println(validateStock(5,10))
    // 5
    println(validateStock(null,10))
    // -1
    println(validateStock(-2,10))
    // -1
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-null-safety-exercise-4"}

|--|--|

```
fun validateStock(requested: Int?, available: Int?): Int {
    val validRequested = requested ?: return -1
    val validAvailable = available ?: return -1

    if (validRequested < 0) return -1
    if (validRequested > validAvailable) return -1

    return validRequested
}

fun main() {
    println(validateStock(5,10))
    // 5
    println(validateStock(null,10))
    // -1
    println(validateStock(-2,10))
    // -1
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-null-safety-solution-4"}

- [Previous step](#)
- [Next step](#)